

CliniqAI Phase 1

Deep Line-by-Line Explanation

Every variable explained
Every function broken down
Every line with examples

Google Cloud Rapid Agent Hackathon · May 2026

Table of Contents

Part 1: Drug Conflict Checker (alert_tool.py)

Part 2: Vision Tool (vision_tool.py)

Part 3: Server Backend (server.py)

Part 4: Variable Tracing

PART 1: Drug Conflict Checker

Understanding alert_tool.py

Concept 1: What is an Allergy Family?

■ LAYER 1 — The Story

Imagine you're allergic to peanuts. Peanuts are in the 'legume' family. Other legumes include lentils, chickpeas, soybeans. If you're allergic to peanuts, you might also be allergic to other legumes. Doctors group them together. In CliniqAI, we do the same with medicines.

■■ LAYER 2 — Technical

Allergy families are dictionaries that group similar drugs:

```
ALLERGY_FAMILIES = { "penicillin": ["amoxicillin", "ampicillin", "augmentin", ...],  
  "cephalosporin": ["cefalexin", "cefuroxime", "cefixime", ...], }
```

Key: Family name (e.g., 'penicillin')

Value: List of drugs in that family

■ LAYER 3 — Example

Example:

```
Patient's allergy: "penicillin" New prescription: "Amoxicillin" Check: Is "amoxicillin" in  
ALLERGY_FAMILIES["penicillin"]? Answer: YES → ALERT!
```

Layer 1: Direct Allergy Check

How Direct Allergy Checking Works

■ LAYER 1 — The Story

A pharmacist checks: 'Is the patient allergic to this exact drug family?' They look at the patient's known allergies and the new prescription, and see if there's a match.

■■ LAYER 2 — Technical

The code loops through 3 things:

1. Each patient allergy
2. Each drug family in the database
3. Each new medicine in the prescription

```
for allergy in patient_allergies: allergy_lower = allergy.lower() for family, drugs in ALLERGY_FAMILIES.items(): if allergy_lower == family or allergy_lower in drugs: for new_name in new_med_names: if any(drug in new_name for drug in drugs): alerts.append({"severity": "HIGH", ...})
```

■ LAYER 3 — Example

Example:

```
Patient allergies: ["penicillin"] New medicines: ["amoxicillin", "paracetamol"] Loop 1: allergy = "penicillin" Loop 2: family = "penicillin", drugs = ["amoxicillin", ...] Loop 3: new_name = "amoxicillin" Check: Is "amoxicillin" in ["amoxicillin", ...]? YES → Add HIGH alert!
```

PART 2: Vision Tool

Understanding vision_tool.py

Image Processing: Image.open(io.BytesIO(image_bytes))

■ LAYER 1 — The Story

Imagine you receive a photo as a stream of binary data (1s and 0s). You need to convert it into something you can actually look at — an image object. That's what this line does.

■ LAYER 2 — Technical

Breaking it down:

`io.BytesIO(image_bytes)` — Convert raw bytes into a file-like object

`Image.open(...)` — Open and validate the image

`try...except` — Catch errors if it's not a valid image

```
try: image = Image.open(io.BytesIO(image_bytes)) except Exception as e: return {"error":  
f"Could not read image file: {str(e)}"}
```

■ LAYER 3 — Example

Example:

```
User uploads: document.txt (NOT an image!) Image.open() throws: UnidentifiedImageError We  
catch it and return: {"error": "Could not read image file: cannot identify image file.  
Please upload a valid JPG or PNG."}
```

PART 3: Server Backend

Understanding server.py

Loading Environment Variables

■ LAYER 1 — The Story

Your API keys (like `GOOGLE_API_KEY`) are secrets. You don't want to hardcode them in your code. Instead, you put them in a `.env` file and load them at runtime.

■■ LAYER 2 — Technical

The process:

1. Create a `.env` file with your secrets
2. Call `load_dotenv()` to read the file
3. Use `os.getenv()` to access the values

```
# .env file GOOGLE_API_KEY=sk-proj-abc123...  
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/cliniquai # In Python load_dotenv()  
MONGODB_URI = os.getenv("MONGODB_URI") GOOGLE_API_KEY = os.getenv("GOOGLE_API_KEY")
```

■ LAYER 3 — Example

Example:

```
After load_dotenv(): MONGODB_URI = "mongodb+srv://user:pass@cluster.mongodb.net/cliniquai"  
GOOGLE_API_KEY = "sk-proj-abc123..." These are now available throughout your code!
```

PART 4: Variable Tracing

Where every variable comes from

Variable	Where it comes from	Example
image_bytes	User uploads file → file.read()	b'\x89PNG...'
extracted	Gemini Vision → extract_from_prompt(prompt)	{'patient_name': 'Ramesh', ...}
patient_name	extracted.get('patient_name')	'Ramesh Gupta'
existing_patient	MongoDB search → find_one()	{_id, name, visits, ...}
visit	Built from extracted data	{date, doctor, medicines, ...}
patient_allergies	Combined list → list(set(...))	['penicillin', 'sulfa']
new_medicines	extracted.get('medicines')	[{name, dose, ...}]
conflict_result	Alert tool → check_drug_conflicts()	{as_alerts, alerts, ...}